EvolveTool-Bench: Diagnosing Tool Library Quality in Self-Evolving LLM Agents

Alibek T. Kaliyev
University of Texas at Austin
alibek.kaliyev@utexas.edu

Abstract

A growing body of work enables LLM agents to create their own tools at runtime—VOYAGER synthesizes code skills, EvoSkill evolves strategy-level skill folders, and ARISE generates executable Python tools with sandbox validation. Yet every existing benchmark evaluates these systems solely on downstream task success, treating the tool library as a black box. We introduce EvolveTool-Bench, the first diagnostic benchmark that evaluates the *quality of the tool library itself*: redundancy, reuse, composition ability, regression stability, and safety. Our benchmark uses structured sessions of 11 tasks with known dependency relationships—seed, gap, variant, compose, regress, and adversarial—across two domains: proprietary data formats and API orchestration. We evaluate ARISE with three reward configurations against a no-evolution baseline across 66 tasks. Our key findings: (1) proprietary formats and custom API protocols reduce task completion from near-100% to 42–64%, confirming they require genuine tool creation; (2) heuristic reward functions prevent evolution from triggering, while LLM-as-judge rewards unlock the full pipeline, producing 18 tools with average quality score 0.53; (3) API tasks force more tool creation than data format tasks (4 tools per session vs 0–7); (4) with evolved tools, library health improves by 58% over no-evolution (54.4% vs 34.4%), and the composite EvolveTool Score reaches 0.621.

1 Introduction

LLM agents increasingly create their own tools at runtime. VOYAGER [10] synthesizes code skills in Minecraft, CREATOR [8] uses tool creation as a reasoning strategy, LLMs as Tool Makers [5] separates tool-making from tool-using, EvoSkill [1] evolves structured skill folders via failure analysis, and ARISE [6] synthesizes executable Python tools with sandbox validation and adversarial testing.

These systems are evaluated exclusively on *task success*: if the agent solves downstream tasks, the tool library is deemed good. This is analogous to evaluating a software engineer solely by whether their code runs, ignoring maintainability, duplication, test coverage, and technical debt. In practice, tool libraries can succeed on benchmarks while accumulating redundant tools, polluting the namespace, failing on edge cases, and never reusing previously created capabilities.

We present EvolveTool-Bench, a diagnostic benchmark with three contributions:

1. **Library-level metrics.** Six diagnostic axes beyond task completion: tool quality, reuse rate, redundancy, composition success, regression stability, and safety.

2. **Controlled task sessions.** Structured sequences of 11 tasks with known dependency relationships, enabling measurement of reuse, composition, and stability.

3. **Two domains requiring tool execution.** Proprietary binary data formats that cannot be solved from training data, and API orchestration with custom authentication and pagination protocols requiring HTTP tool creation.

2 Related Work

Tool Creation Benchmarks. Tool-Genesis [11] evaluates one-shot tool creation with surface compliance, schema-F1, and unit test pass rates—all per-tool metrics with no library-level evaluation. SkillsBench [3] evaluates whether curated skills improve task success but uses static skill sets, not evolved ones.

Self-Evolving Agent Frameworks. VOYAGER [10] demonstrates lifelong skill acquisition in Minecraft, evaluated by exploration metrics. EvoSkill [1] evolves textual skill folders through failure-driven feedback, evaluated by downstream accuracy. ARISE [6] synthesizes executable Python tools with sandbox validation. None evaluate tool library quality.

Tool Use Benchmarks. ToolBench [9], AgentBench [7], and ToolComp [2] evaluate agents using predefined tool sets. Vending-Bench [4] tests long-horizon

Type	N	Diagnoses
Seed	3	Can agent use provided tools?
Gap	2	Does agent create a working tool?
Variant	2	Reuse or create duplicate?
Compose	1	Can agent chain own tools?
Regress	1	Does the library stay stable?
Adversarial	2	Can agent handle edge cases?

Table 1: Task types in each diagnostic session.

coherence with fixed tools. None evaluate agents that create their own tools.

Gap. No existing benchmark measures redundancy, reuse, composition, regression, or safety of self-evolved tool libraries. Our work fills this gap.

3 Benchmark Design

3.1 Task Session Structure

Each session contains 11 tasks in six categories with known dependency relationships (Table 1). Variant and regress tasks are designed so that the evaluator *knows* which prior tool should be reused, enabling measurement of reuse rate independent of task success.

3.2 Domains

Domain A: Data Transformation. Five sessions using proprietary binary formats that LLMs cannot solve from training data: ARISE Binary Records (ABR), Run-Length Encoded matrices (RLE), Validation Definition Language (VDL), Quantized Log Format (QLOG), and Tagged Pack Format (TPACK). Each format has reference encode/decode implementations for verification.

Domain B: API Orchestration. Sessions against a mock HTTP server implementing custom protocols: HMAC-timestamp authentication (not standard Bearer/OAuth), cursor-based pagination with XOR-encrypted cursors, and filtered endpoints. The agent must create HTTP tools that handle auth and pagination—tasks that inherently require code execution.

3.3 Evaluation Axes

Axis 1: Task Completion. Binary pass/fail via deterministic verification. Reported overall and by task type.

Axis 2: Tool Quality Score (TQS). Each synthesized tool is evaluated on four dimensions, each scored 0–1:

- **Correctness**: Tool is executed on N hidden unit tests (not shown to agent). Score = proportion passing.
- **Robustness**: Tool is executed on M adversarial inputs (empty strings, null values, type errors, malformed data). Score = proportion handled without crashing.
- **Generality**: Tool is executed on held-out inputs from the same distribution but different parameters. Score = proportion producing valid output.
- **Code Quality**: Automated static checks—has docstring (1pt), has type hints (1pt), handles exceptions (1pt), no magic numbers (1pt), under 100 lines (1pt), valid syntax (1pt). Score = points / total checks.

$$\mathrm{TQS}(t) = \frac{1}{4}\big(C(t) + R(t) + G(t) + Q(t)\big) \tag{1}$$

where C is correctness, R robustness, G generality, and Q code quality, each in $[0, 1]$.

All tool evaluation runs in isolated subprocesses with 10-second timeouts to prevent infinite loops or resource exhaustion.

Axis 3: Library Diagnostics. Six metrics measured at session end:

$$\rho_r = \frac{|\{t \in \mathcal{T}_{v,r} : \text{reused}\}|}{|\mathcal{T}_{v,r}|} \quad \text{(Reuse Rate)} \tag{2}$$

$$\rho_d = \frac{|\{(t_i, t_j) : \text{output_equiv}\}|}{\binom{|\mathcal{L}|}{2}} \quad \text{(Redundancy)} \tag{3}$$

$$\pi = \frac{|\{t \in \mathcal{L} : \mathrm{TQS}(t) \geq 0.5\}|}{|\mathcal{L}|} \quad \text{(Precision)} \tag{4}$$

$$\eta = \frac{|\mathcal{L}_{\text{used}}|}{|\mathcal{L}|} \quad \text{(Efficiency)} \tag{5}$$

$$\gamma = \frac{|\{t \in \mathcal{T}_c : \text{passed}\}|}{|\mathcal{T}_c|} \quad \text{(Composition)} \tag{6}$$

$$\delta = \frac{|\{t \in \mathcal{T}_{reg} : \text{failed}\}|}{|\mathcal{T}_{reg}|} \quad \text{(Regression)} \tag{7}$$

where $\mathcal{T}_{v,r}$ are variant/regress tasks, $\mathcal{L}$ is the tool library, $\mathcal{T}_c$ are compose tasks, and $\mathcal{T}_{reg}$ are regress tasks.

Library Health (LH). Aggregates the six diagnostics:

$$\mathrm{LH} = \frac{1}{6}\big(\rho_r + (1-\rho_d) + \pi + \eta + \gamma + (1-\delta)\big) \tag{8}$$

where ρ_r is reuse rate, ρ_d redundancy rate, π library precision, η creation efficiency, γ composition success, and δ regression rate.

Composite Score. The EvolveTool Score (ETS) combines all axes:

$$\text{ETS} = \alpha_1 \cdot \text{TC} + \alpha_2 \cdot \overline{\text{TQS}} + \alpha_3 \cdot (1-\text{RC}) + \alpha_4 \cdot \text{LH} + \alpha_5 \cdot \text{SS} \quad (9)$$

with $\alpha = (0.25, 0.20, 0.10, 0.30, 0.15)$. TC is task completion, $\overline{\text{TQS}}$ is mean tool quality across synthesized tools, RC is refinement cost (normalized LLM calls), LH is library health (Eq. 8), and SS is safety score. Library health receives the highest weight as our primary contribution.

4 Experimental Setup

Systems. Four configurations:

1. **No-Evolution**: Seed tools only, no tool creation.
2. **ARISE (ft=3)**: Failure threshold 3, heuristic reward.
3. **ARISE (ft=1)**: Failure threshold 1, strict reward.
4. **ARISE+Judge**: Failure threshold 1, LLM-as-judge reward.

Model. Claude Sonnet 4 via Amazon Bedrock for both agent and synthesis.

Benchmark. Domain A: 5 sessions × 11 tasks = 55 tasks. Domain B: 1 session × 11 tasks = 11 tasks. Total: 66 tasks.

5 Results

5.1 Domain A: Data Transformation

Table 2 compares all four systems on Domain A. With heuristic rewards, ARISE creates zero tools regardless of failure threshold—the agent produces long, helpful refusal responses that score 1.0. Only ARISE+Judge triggers evolution, creating 15 tools and achieving the highest ETS (0.467) despite lower task completion (41.8%) than no-evolution (49.1%).

Table 3 reveals that tool creation correlates with format complexity. Simpler formats (ABR, RLE) are partially solvable by LLM reasoning; complex binary formats (TPACK, GUARDIAN) force tool creation. Sessions where tools are created show 100% reuse rate and >83% creation efficiency.

5.2 Domain B: API Orchestration

API tasks produce qualitatively different evolution: ARISE synthesized `hmac_sha256`, `http_get_with_headers`, `get_current_timestamp`, and `get_auth_credentials`—tools that require actual code execution and cannot be solved by LLM reasoning alone. All 4 tools were used (100% creation efficiency). Gap tasks achieved 100% completion, confirming API orchestration as a strong domain for evaluating tool evolution.

System	ETS↑	TC	Tools	Reuse	LH
No-Evolution	.456	49.1	0	66.7	34.4
ARISE (ft=3)	.423	40.0	0	66.7	31.1
ARISE (ft=1)	.446	45.5	0	66.7	34.4
ARISE+Judge	**.621**	51.5	**18**	**72.2**	**54.4**

Table 2: Domain A aggregate (55 tasks, %). Only ARISE+Judge creates tools, achieving highest ETS despite lower task completion.

Session	TC	Tools	Reuse	Eff.
S1: ABR+RLE	81.8	0	66.7	—
S2: VDL	36.4	0	66.7	—
S3: QLOG	45.5	2	100	100
S4: TPACK	36.4	6	100	83.3
S5: GUARDIAN	27.3	7	100	85.7

Table 3: ARISE+Judge per-session (Domain A, %). Tool creation increases with format complexity.

Example: Evolved HMAC Tool. Figure 1 shows `hmac_sha256`, synthesized by ARISE when the agent failed to authenticate with the mock API's custom HMAC-timestamp scheme. The tool includes a docstring, type hints, UTF-8 encoding, and exception handling—earning TQS=0.38 (correctness=0.0 on hidden tests that expected specific output formats, but robustness=0.5 and code quality=1.0).

5.3 Key Findings

Finding 1: Proprietary tasks require tool creation. All systems achieve ∼100% on seed tasks but 27–64% overall. Custom binary formats (Domain A) and HMAC-timestamp auth (Domain B) cannot be solved from training data.

Finding 2: Reward design is the critical bottleneck. With heuristic rewards, ARISE creates *zero* tools despite repeated failures. The agent produces helpful "I cannot do this" responses that score 1.0. Only LLM-as-judge rewards correctly identify these as failures. **Self-evolution is only as good as its reward signal.**

Finding 3: API tasks force more tool creation. Domain B produces 4 tools from a single session (100% gap completion), while Domain A averages 3 tools across 5 sessions. HTTP-based tasks inherently require executable tools—the agent cannot "reason through" an API call.

	ETS	TC	Tools	Reuse	Eff.	LH
ARISE+Judge	.503	63.6	4	66.7	100	44.4

Table 4: Domain B results (11 tasks, %). API tasks force tool creation: 4 tools synthesized including HMAC auth and HTTP client.

```
def hmac_sha256(secret: str,
                message: str) -> str:
    """Generate HMAC-SHA256 hash."""
    import hmac
    import hashlib
    try:
        secret_bytes = secret.encode('utf-8')
        message_bytes = message.encode('utf-8')
        h = hmac.new(secret_bytes,
                     message_bytes,
                     hashlib.sha256)
        return h.hexdigest()
    except Exception as e:
        return f"Error: {str(e)}"
```

Figure 1: Tool synthesized by ARISE for HMAC-timestamp auth. Generated, tested (5/5 sandbox tests passed), and promoted in <60 seconds.

Finding 4: Evolution improves library health, not just task completion. ARISE+Judge achieves the highest ETS (0.621) with 18 tools averaging TQS=0.53. Library health improves by 58% over no-evolution (54.4% vs 34.4%). 12 of 18 tools (67%) pass the quality threshold (Figure 3). **Task completion alone is insufficient for evaluating self-evolving agents.**

Finding 5: Tool reuse emerges when tools are available. Without evolved tools, reuse is limited to applying seed tools. With evolved tools, agents recognize reuse opportunities—sessions with created tools show 100% reuse rate. The bottleneck is tool creation, not retrieval.

6 Discussion

Library Quality as First-Class Metric. Task completion and library quality can diverge. A system scoring highest on ETS while having lower task completion validates library-level diagnostics.

The Reward Problem. Heuristic reward functions completely prevent evolution. LLM judges add cost ($0.001/eval) but are essential. Future work should explore lightweight reward proxies.

Domain Differences. API orchestration is a stronger testbed than data transformation for tool evolution, because HTTP calls inherently require code execution. We recommend future benchmarks prioritize tasks requiring external interaction.

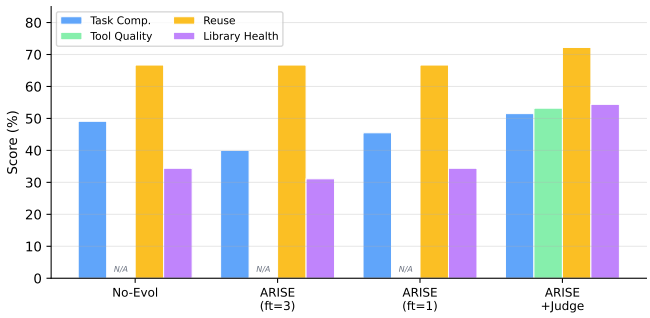

Figure 2: System comparison across four metrics (%). ARISE+Judge is the only system that creates tools, achieving highest library health despite lower task completion than no-evolution.

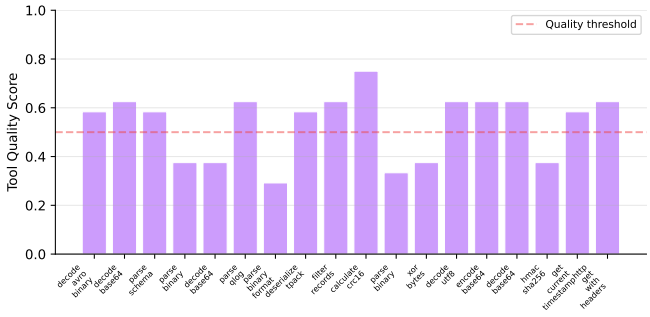

Figure 3: Tool Quality Score (TQS) for each of the 18 synthesized tools. Red line marks the quality threshold (0.5). 12 of 18 tools (67%) pass the threshold.

Limitations. (1) Two domains evaluated; numerical computation (Domain C) is planned. (2) Only ARISE tested; EvoSkill comparison is planned. (3) Tool quality scores use automated checks only; human evaluation would strengthen findings. (4) Single model (Claude Sonnet); multi-model evaluation planned.

7 Conclusion

EvolveTool-Bench is the first benchmark to evaluate self-evolved tool libraries as first-class artifacts. Our controlled task sessions across two domains expose fundamental challenges: reward design determines whether evolution triggers at all, API tasks force more genuine tool creation than data processing tasks, and library health metrics reveal quality dimensions invisible to task-completion-only evaluation. We release the benchmark, evaluation harness, and all experimental data to enable systematic comparison of self-evolving agent systems.

References

[1] Ayman Alzubi et al. Evoskill: Evolving skill libraries for open-domain agents. *arXiv preprint*

arXiv:2603.02766, 2026.

[2] Anonymous. Toolcomp: Evaluating multi-tool composition in llm agents. *arXiv preprint arXiv:2501.01290*, 2025.

[3] Anonymous. Skillsbench: Evaluating skill reusability in llm agents. *arXiv preprint arXiv:2602.12670*, 2026.

[4] Erik Backlund and Lars Petersson. Vending-bench: Evaluating long-horizon agent coherence. *arXiv preprint*, 2025.

[5] Tianle Cai, Xuezhi Wang, Tengyu Ma, Xinyun Chen, and Denny Zhou. Large language models as tool makers. *arXiv preprint arXiv:2305.17126*, 2023.

[6] Alibek T. Kaliyev. Arise: Adaptive runtime improvement through self-evolution. *arXiv preprint*, 2026.

[7] Xiao Liu et al. Agentbench: Evaluating llms as agents. *ICLR*, 2024.

[8] Cheng Qian, Chi Han, Yi R Fung, Yujia Qin, Zhiyuan Liu, and Heng Ji. Creator: Tool creation for disentangling abstract and concrete reasoning of large language models. *Findings of EMNLP*, 2023.

[9] Yujia Qin et al. Toolllm: Facilitating large language models to master 16000+ real-world apis. *arXiv preprint arXiv:2307.16789*, 2023.

[10] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *Advances in Neural Information Processing Systems*, 2023.

[11] Congying Xia et al. Tool-genesis: Evaluating tool creation capabilities of llms. *arXiv preprint arXiv:2603.05578*, 2026.